

From Tokens to Reasoning to Intelligent Agentic AI Systems: Building a Sovereign Foundational AI Stack

Piyush Sawarkar

Founding Member & Tech Lead Text Foundation Models @ BharatGen, India's Sovereign AI National Mission 
IIT Bombay Alumnus

Mixture of Experts • Activate & BharatGen • IIT Bombay • September 11, 2026



BharatGen

GenAI for Bharat, by Bharat



Funded by



विज्ञान एवं प्रौद्योगिकी विभाग
DEPARTMENT OF
SCIENCE & TECHNOLOGY



INDIAai
A MEITY INITIATIVE

Academic
Consortium



सिद्धिमूलं प्रबन्धनम्
भा. प्र. सं. इन्दौर
IIM INDORE



Infrastructure
Partners

neysa



Backed by
the govt,
built with
India's best



OUR MODEL SUITE

Text. Voice. Documents. Across Indian languages

TEXT · LLM

Param Series

Multilingual · Reasoning, code, math, and tool use · 22 Indian languages + English · Multiturn, instruction-tuned

SPEECH · ASR

Shrutam Series

LLM-powered · MoE projector · 12 Indian languages released · Realtime transcription · Code-switching

VISION · DOCUMENT VLM

Patram Series

Vision transformer · Document QA and information extraction · Multi-domain document support · Table/chart understanding

SPEECH · TTS

Sooktam Series

Cross-lingual voice cloning · CLS · 11 Indian languages + English released · Zero-shot voice generation · Natural, human-sounding output

Talk Overview

What it takes to build a Foundational AI from Scratch?

The Model Development Life Cycle

Training BharatGen's Foundation Model – A Case Study!

The LLM Foundations

Data Curation -> Tokenizer -> Model Architecture -> Pretraining

Graduating the Model to be Conscious: *Imitation to Reasoning to Agentic AI Capabilities*

Reinforcement Learning

Intelligent Agentic AI Systems

Why building from Scratch? – A Good Question to Think!

Talk Overview

What it takes to build a Foundational AI from Scratch?

The Model Development Life Cycle

Training BharatGen's Foundation Model – A Case Study!

The LLM Foundations

Data Curation -> Tokenizer -> Model Architecture -> Pretraining

Graduating the Model to be Conscious: *Imitation to Reasoning to Agentic AI Capabilities*

Reinforcement Learning

Intelligent Agentic AI Systems

Why building from Scratch? – A Good Question to Think!

What it takes to build a Foundational AI from Scratch?



Talk Overview

What it takes to build a Foundational AI from Scratch?

The Model Development Life Cycle

Training BharatGen's Foundation Model – A Case Study!

The LLM Foundations

Data Curation -> Tokenizer -> Model Architecture -> Pretraining

Graduating the Model to be Conscious: *Imitation to Reasoning to Agentic AI Capabilities*

Reinforcement Learning

Intelligent Agentic AI Systems

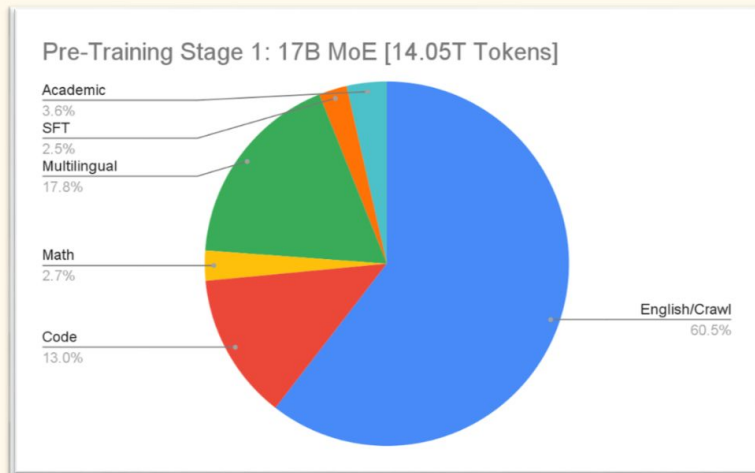
Why building from Scratch? – A Good Question to Think!

Training Dataset details for the Param 17B Foundation Model

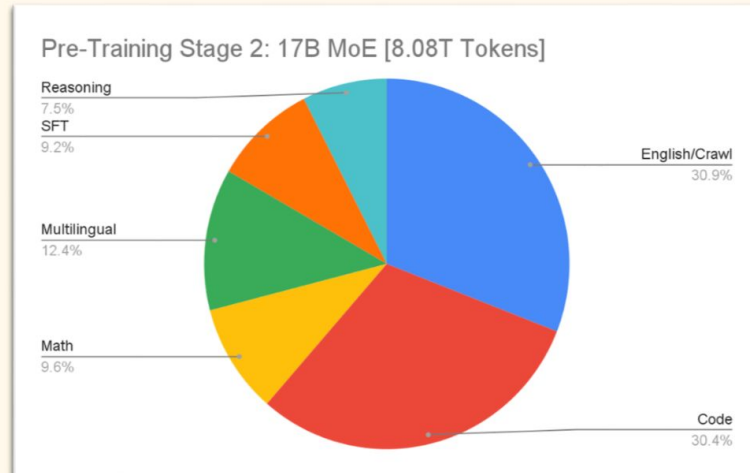
PARAM 2 17B MoE - DATA DETAILS

Dataset Preparation Details: Multi-Stage Pretraining on 22.13 Trillion Tokens

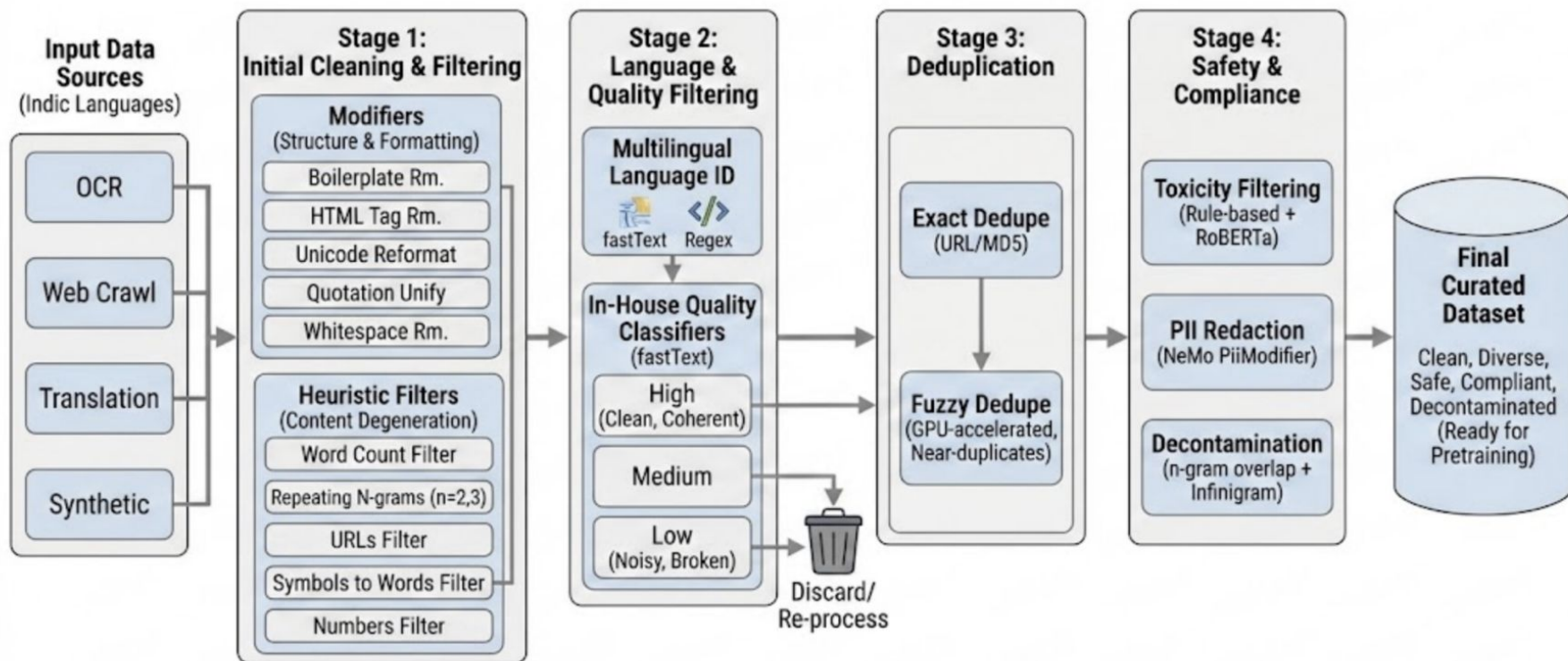
Pre-Training Stage 1 [14.05T]



Pre-Training Stage 2 [8.08T]



Data Curation Process



The Tokenizer

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🖐

Sequences of characters commonly found next to each other may be grouped together: 1234567890



After Tokenization

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🖐

Sequences of characters commonly found next to each other may be grouped together: 1234567890

The Tokenizer

A tokenizer converts raw text into numbers the model can process. It breaks words into chunks called tokens. The model never sees the original text, it only sees sequences of numbers representing tokens.

What is a token?

A token is not necessarily a word. It is the basic unit the model is trained on. OpenAI's tokenizer, for example, splits "Bombay" into two tokens: "Bomb" + 'ay". BharatGen's tokenizer treats "Bombay" as one token, because it was trained on Indian data and "Bombay" is a well-known place.

Word	OpenAI tokenizer	BharatGen tokenizer
Bombay	2 tokens: "Bomb" + 'ay" → charged 2x	1 token: "Bombay" → charged 1x
Hindi word (e.g., नमस्ते)	May split into 6–7 tokens (byte-level fallback)	1–2 tokens (trained on Indian scripts)

The Tokenizer

Fertility: the key metric

Fertility is the ratio of tokens to words. Lower is better: it means the tokenizer is more efficient at representing the language.

Fertility (English) ~1.4 tokens per word (industry standard)	Fertility (Indian langs, global models) 6–7x tokens per Indian-language word	Fertility (Indian langs, BharatGen) 1–3x tokens per Indian-language word
---	---	---

Why fertility matters commercially: API pricing is charged per token. If a global model charges \$10 per 1M output tokens and uses 6–7 tokens for every Indian-language word, an Indian enterprise using it in Hindi is paying 4–5x more than they would with BharatGen's model for the same output. This is a direct cost argument when talking to clients.

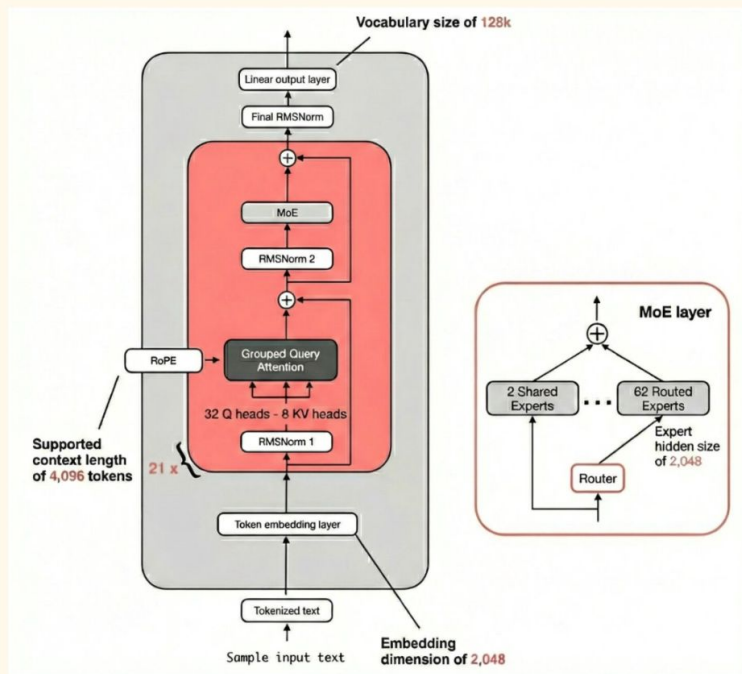
BharatGen's Tokenizer Fertility

Language	AdaptMix (Param)	Qwen	LLaMA	Nemotron Mistral	Nemotron Mini	Sarvam-M	DeepSeekv3	Gemma 27B	Airavata
Assamese	1.93	7.18	8.06	4.24	4.58	4.24	3.59	2.68	8.94
Bengali	1.90	6.92	7.85	2.93	2.65	2.93	2.89	1.74	8.20
English	1.47	1.36	1.35	1.37	1.35	1.37	1.33	1.35	1.58
Gujarati	2.03	8.53	9.54	3.59	15.17	3.59	4.84	2.39	14.14
Hindi	1.43	4.66	2.65	1.97	1.77	1.97	2.92	1.38	1.80
Kannada	2.30	11.08	13.81	3.82	4.02	3.82	5.83	3.15	19.35
Maithili	1.73	4.67	2.85	2.53	2.28	2.53	3.28	1.90	2.45
Malayalam	2.60	13.30	16.00	4.88	4.71	4.88	7.83	3.39	11.38
Marathi	1.87	6.46	3.86	3.14	2.62	3.14	4.15	1.94	3.31
Nepali	1.70	6.28	3.61	3.04	2.32	3.04	4.07	2.06	3.05
Oriya	1.95	12.92	15.91	1.71	17.24	17.23	7.26	4.60	17.29
Punjabi	1.61	7.39	7.88	3.12	12.70	3.12	4.51	2.74	10.84
Sanskrit	2.57	8.00	4.75	4.26	4.32	4.26	4.95	3.36	4.66
Sindhi	1.67	3.09	2.99	2.65	2.83	2.65	2.98	2.14	5.04
Tamil	2.35	9.75	11.89	3.71	3.57	3.71	4.88	2.42	10.50
Telugu	2.34	11.45	13.30	3.90	3.77	3.90	5.99	2.93	19.51
Average	1.97	7.69	7.89	3.18	5.37	4.15	4.46	2.51	8.88

Our Current Flagship Model: Param-17B Mixture of Experts LLM

Param-2-17B-MOE-A2.4B: High-Efficiency Multilingual Powerhouse

State-of-the-art Mixture-of-Experts (MoE) model with 17B total parameters (2.4B active per token), combining massive knowledge capacity with fast, efficient inference, trained on ~22T tokens across 64 experts. Supports 22 Indian languages with strong code, math, and tool-use reasoning, optimized for culturally grounded, region-specific tasks and inclusive representation.



Languages supported		
• Assamese	• Kannada	• Punjabi
• Bengali	• Kashmiri	• Sanskrit
• Bodo	• Konkani	• Santali
• Dogri	• Maithili	• Sindhi
• Gujarati	• Malayalam	• Tamil
• Hindi*	• Manipuri	• Telugu
• Marathi	• Nepali	• Urdu
• Odia (Oriya)		

Now available for access through:

- **AIKosh:** [Link](#)
- **Hugging Face:** [Link](#)

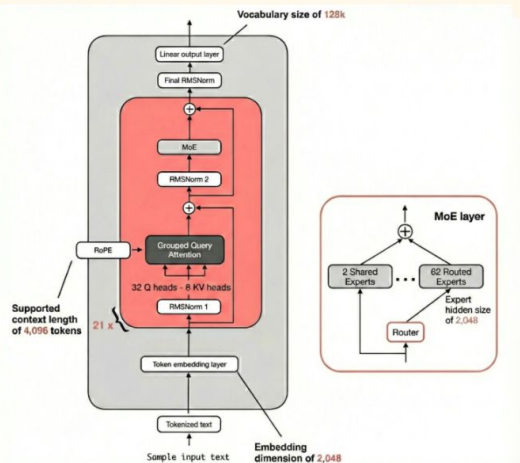
Our Current Flagship Model: Param-17B Mixture of Experts LLM

PARAM 2 17B MoE - Hybrid MoE: Efficiency & Specialization

Architectural Innovation

Employs a unique **Hybrid MoE** approach to solve the "instability" often found in sparse models:

- Shared Experts:** Two experts remain **always active** to maintain stable cross-lingual representations and smooth code-switching.
- Routed Experts:** Six experts dynamically selected per token to handle specialized tasks and specific language nuances.
- Dense Pre-processing:** A dense layer transforms raw inputs into structured representations before MoE, ensuring better training stability.



Metric	Value	Metric	Value
Total Parameters	17B	Active Params/Token	2.4B
Total Experts	64	Top-K Experts	6 (Routed) + 2 (Shared)
Context Length	4096 Tokens	Vocabulary Size	128,000
Attention	Grouped Query (GQA)	Precision	bf16-mixed
Hidden Layers	21	Activation	SiLU / RMSNorm

Forward Looking: A long-context variant supporting up to **128k tokens** currently in development for long-form document processing.

So are we done with Training? Apparently, we've just begun!

STAGE 1

Pre-training: knowledge without behaviour

The only objective

$$\mathcal{L} = - \sum_t \log p(\mathbf{x}_t | \mathbf{x}_{<t})$$

- Cross-entropy at every token: given a context, predict what comes next.
- Trained on trillions of tokens — the knowledge is unquestionably in there.
- But it lives spread across a probability distribution in hyper-dimensional space, not as callable behaviour.

✓ What you get

- World knowledge, language, code, facts
- Raw completion ability: continue any context plausibly

✗ What you don't get

- Instruction adherence — it completes, it does not obey
- Comprehension of a task, planning, building a blueprint
- All the "fancy stuff" you associate with ChatGPT / Claude

RECAP

1. Data collect + curate	2. Tokenizer language encoding	3. Architecture model design	4. Pretraining largest stage	5. Post-training SFT + RL	6. Evaluation benchmarks + linguistic
------------------------------------	--	--	--	-------------------------------------	--

Talk Overview

What it takes to build a Foundational AI from Scratch?

The Model Development Life Cycle

Training BharatGen's Foundation Model – A Case Study!

The LLM Foundations

Data Curation -> Tokenizer -> Model Architecture -> Pretraining

Graduating the Model to be Conscious: *Imitation to Reasoning to Agentic AI Capabilities*

Reinforcement Learning

Intelligent Agentic AI Systems

Why building from Scratch? – A Good Question to Think!

Off-Policy vs On-Policy Learning

THE CORE DISTINCTION

Off-policy imitates. On-policy experiences.

OFF-POLICY · pre-training, SFT, off-policy distillation

Learn from someone else's trajectories

- You force another distribution onto the model; the obedient student imitates it.
- Great for breadth: new formats, new domains, new skills.
- Fragile out of distribution — the Pythagoras failure lives here.

ON-POLICY · reinforcement learning

Learn from your own rollouts

- The model generates, gets rewarded, and updates on its own behaviour.
- Don't hand it the solved equation — let it think harder from knowledge it already has from pre-training.
- Like solving problems at home without peeking at the back pages of the book.

Mnemonic: SFT expands what the model can express. RL sharpens what it can reliably do.

Provenance Mapping key to unlocking and sustaining “Performance Climb!”

BEFORE RL CAN WORK

RL needs in-context learning — and ICL comes from scale

ICL is emergent

In-context learning builds up over training; empirically it strengthens with scale. Without it, on-policy exploration has nothing to steer with.

Two ways to scale

More parameters (a bigger brain) — or more FLOPs on the same brain. Both grow capability; only one grows your serving bill forever.

Overfitting? Please.

Success stories of SLMs over Trillions of Tokens of Training exists — performs good on everyday, and agentic tasks. If you manage to overfit an LLM, that itself is an achievement!

The efficiency thesis: you don't need an Einstein's brain if you learn efficiently. Parameter scaling helps — efficient learning substitutes for it.

Letting the Model Exercise its own Knowledge! – via RL

The model knows a lot. But can it think?



PRACTICE

VERIFICATION

TRAJECTORIES

TOOLS

AGENTS

LEARNING THROUGH PRACTICE & DECISION-MAKING

On-Policy Learning through Reinforcement Learning RL

One Model, Every Capability

Foundation-scale RL across a dozen expert domains — the capability surface we optimise Param for



Mathematical Reasoning

Competition-grade problem solving & proof writing



STEM & Scientific Reasoning

Graduate-level science, research QA & critical reasoning



Coding

From single functions to multi-file repositories



Agentic & Software Engg.

Planning, executing and fixing real pull requests



Search & Tool Use

Multi-hop research with live tools & browsing



Terminal & Environment Use

Operating shells, sandboxes and real workflows



Instruction Following & Writing

Precise, faithful, creative generation



Multilingual Intelligence

22 Indian languages + English, code-switched



Safety & Abstention

Knowing what to answer — and what not to

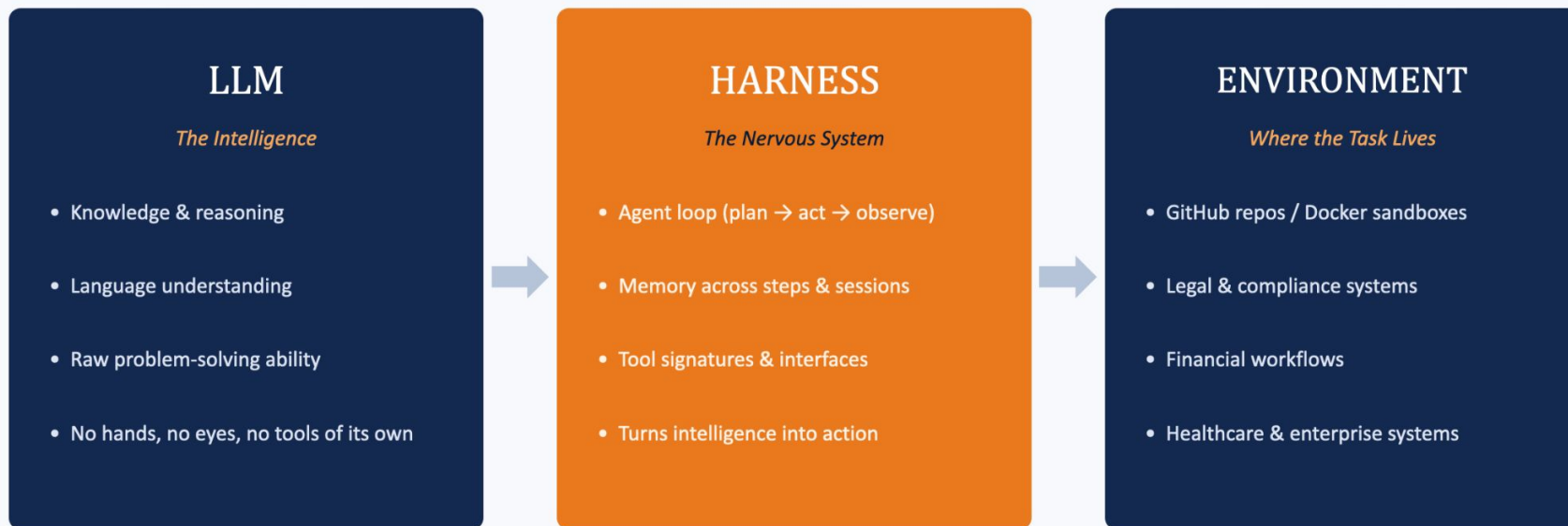
***UP UNTIL NOW,* Imitation to Reasoning to Agentic Capabilities**

***NEXT,* Intelligent Agentic AI Systems**

Graduating LLMs to Intelligent Agent AI Systems

Intelligence Alone Doesn't Act

The Harness is what connects a model's intelligence to a real environment and a real task



The environment changes with the user. The harness is what lets the same model act inside any of them.

Talk Overview

What it takes to build a Foundational AI from Scratch?

The Model Development Life Cycle

Training BharatGen's Foundation Model – A Case Study!

The LLM Foundations

Data Curation -> Tokenizer -> Model Architecture -> Pretraining

Graduating the Model to be Conscious: *Imitation to Reasoning to Agentic AI Capabilities*

Reinforcement Learning

Intelligent Agentic AI Systems

Why building from Scratch? – A Good Question to Think!

Why building from scratch?

01 CONTROL

data • architecture • training • post-training

02 REPRESENTATION

languages • domains • context

03 SOVEREIGNTY

compute • weights • deployment • iteration

04 RESEARCH

push the frontier, not only consume it

FROM CONSUMING INTELLIGENCE → TO BUILDING INTELLIGENCE

Thank You